

Automated Classification of Failed LLM-Generated REST API Test Cases

Scope Description

Selen Öznur

University of Antwerp

July 2026

1 Context

REST APIs are the backbone of modern cloud-native systems. Testing them comprehensively is challenging due to the vast input space, diverse endpoint behaviors, and evolving specifications. Recent work has explored the use of Large Language Model (LLM)-driven agents to automatically amplify existing test suites.

A comparative study by Besjes et al. [1] evaluated both single-agent and multi-agent LLM workflows for REST API test amplification across five cloud applications: Spotify, Google Drive, PetStore, VAmPI, and Restful-Booker. While the results were promising in terms of structural API coverage, a significant portion of the generated test cases failed. These failures were manually analyzed and categorized by the research team into five categories:

- **Semantically Incorrect:** the test logic is fundamentally inconsistent with the endpoint's intended behavior
- **Wrong Assertion:** the request is valid, but the expected outcome is incorrect
- **Missing Information:** essential information is unavailable in the test environment or API documentation
- **Wrong Input:** the test provides invalid, malformed, or incomplete input values
- **Bug:** the test exposes behavior that contradicts the intended API behavior as defined by the OpenAPI specification

This manual classification process produced a labeled dataset of failed test cases across multiple systems and agent configurations, forming the basis of this research.

2 Problem Statement

Manual classification of failed LLM-generated test cases is labor-intensive and does not scale. As agentic systems generate hundreds or thousands of test cases per run, the ability to automatically understand *why* a test fails becomes critical for:

- Prioritizing which failures deserve developer attention
- Identifying systematic weaknesses in the LLM agent pipeline
- Enabling automated repair strategies targeted at specific failure types

Prior studies have primarily focused on LLM-based test generation, test amplification, test repair, and the analysis of errors arising when REST APIs are used as LLM tools [3, 4, 6]. The automated classification of failed LLM-generated REST API test cases according to an existing

manually defined taxonomy remains comparatively underexplored. This study investigates an approach to this classification problem.

3 Research Questions

RQ1: To what extent can an LLM-based classifier automatically reproduce the manual failure categorization of the existing labeled dataset?

RQ2: How does the inclusion or exclusion of different input features, such as test code, API specification, error message, and HTTP method, affect failure classification performance?

RQ3: How does classification performance vary across different REST APIs and failure categories?

RQ4: How does failure classification performance vary across selected LLMs under consistent prompting and evaluation conditions?

4 Approach

The research will use the existing labeled dataset from the Test Amplification reproduction package as reference labels [2]. The proposed approach consists of the following steps:

1. **Dataset analysis:** Explore the distribution of failure categories across systems and agent configurations. Identify class imbalance and inter-system variation.
2. **Input preparation:** Extract and structure the available information from the reproduction package for each failed test case, including the test code, relevant API endpoint specification, execution error message, and HTTP method. System and agent-configuration metadata will be retained for API-level and configuration-level analysis, but will not be used as direct classification inputs.
3. **LLM-based classification:** Implement an automated classification pipeline that accesses two selected LLMs through their programming APIs. The selected model names, versions, prompts, and inference parameters will be documented to support reproducibility. Zero-shot, few-shot, and definition-guided prompting strategies will be evaluated to classify each failed test case into one of the five failure categories. [5]
4. **Feature contribution analysis:** Conduct controlled feature ablation experiments to evaluate the contribution of each input source. A full-input configuration containing the test code, API specification, execution error message, and HTTP method will be compared with configurations in which one information source is removed at a time. The model, prompt template, dataset split, and inference parameters will be kept constant across configurations. Feature contribution will be estimated from the resulting change in macro-F1 and per-category F1-scores.
5. **Cross-model comparison:** Compare GPT-4o mini and Claude Haiku 4.5 using the same dataset, prompt templates, input configurations, few-shot examples, and evaluation procedure. Fixed model versions will be used and documented to support reproducibility. The comparison will examine overall performance, per-category performance, and recurring misclassification patterns across the two models.
6. **Evaluation:** Use macro-F1 as the primary classification metric because the failure categories may be imbalanced. Report accuracy, balanced accuracy, Cohen’s kappa, per-

category precision, recall, and F1-score as complementary measures. Use confusion matrices to identify categories that are frequently confused. Compare feature configurations and selected LLMs using changes in macro-F1 and per-category F1-scores, and report results separately for each REST API. Finally, conduct a qualitative analysis of misclassified cases to identify recurring error patterns.

5 Expected Contributions

1. A prototype and empirical evaluation of an automated approach for classifying failure modes in LLM-generated REST API test cases.
2. An empirical assessment of the contribution of different input sources to failure classification performance.
3. A comparison of selected LLMs under consistent prompting and evaluation conditions.
4. Insights into recurring failure patterns in the studied agentic LLM test amplification workflows, which may guide future improvements to agent design.

6 Relation to Existing Work

This research extends the REST API test amplification study by Besjes et al. [1], which produced and manually labeled the dataset used in this work. The present study investigates how this classification process can be automated.

Related work by Bandlamudi et al. [6] introduced an error taxonomy for REST APIs used as LLM tools. While their focus is API readiness for agent-based tool use, this study focuses on classifying failed LLM-generated REST API test cases. The work is also situated within the broader area of LLM-based software testing [3].

7 Preliminary Planning

Phase	Activity	Period
1	Literature review + dataset familiarization	July 13–17
2	Dataset analysis + feature engineering	July 17–31
3	LLM classifier design + experiments	August 1–25
4	Evaluation + qualitative analysis	August 25 – September 5
5	research writing	Throughout / finalize by September 11

References

- [1] J. Besjes, R. Nooyens, T. Bardakci, M. Beyazit, and S. Demeyer, *Agentic LLMs for REST API Test Amplification: A Comparative Study Across Cloud Applications*. arXiv preprint arXiv:2510.27417, 2025.
- [2] *Reproduction Package – A Comparative Study of Agentic LLMs for REST API Test Amplification*. Zenodo, Version 1, 2026.

- [3] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, *Software Testing with Large Language Models: Survey, Landscape, and Vision*. IEEE Transactions on Software Engineering, vol. 50, no. 4, pp. 911–936, 2024.
- [4] R. Nooyens, T. Bardakci, M. Beyazit, and S. Demeyer, *Test Amplification for REST APIs via Single and Multi-Agent LLM Systems*. International Conference on Testing Software and Systems, pp. 161–177, 2025.
- [5] M. A. Hannan, R. Ni, C. Zhang, L. Jia, R. Mangal, and C. S. Pasareanu, *On the Difficulty of Selecting Few-Shot Examples for Effective LLM-based Vulnerability Detection*. Workshop on LLM Assisted Security and Trust Exploration (LAST-X), 2026.
- [6] J. Bandlamudi, R. Chaudhuri, N. Gantayat, S. Ghosh, K. Mukherjee, P. Agarwal, R. Sindhgatta, and S. Mehta, *A Framework for Testing and Adapting REST APIs as LLM Tools*. arXiv preprint arXiv:2504.15546, 2025.